Supplemental Material: Data Free Training of Diffusion Models for Lattice Field Theories

Dennis Grinwald,[1,2,*] Ankur Singha,[1,2,†] Khaled Kahouli,[1,2] Karl Jansen,[3,4] and Shinichi Nakajima[1,2,5,‡]

[1] *Berlin Institute for the Foundations of Learning and Data (BIFOLD), Germany*
[2] *Technische Universität Berlin, Germany*
[3] *Deutsches Elektronen-Synchrotron (DESY), Platanenallee 6, 15738 Zeuthen, Germany*
[4] *Computation-Based Science and Technology Research Center, The Cyprus Institute, Nicosia, Cyprus*
[5] *RIKEN Center for Advanced Intelligence Project (AIP), Japan*

(Dated: October 13, 2026)

This Supplemental Material collects the background, derivations and implementation detail omitted from the Letter. Equation and reference numbering is independent of the Letter.

I. CRITICAL SLOWING DOWN

Markov-chain Monte Carlo generates a sequence of configurations $\boldsymbol{\phi}^{(1)}, \ldots, \boldsymbol{\phi}^{(N)}$ whose successive elements are correlated. The normalized autocorrelation function at the Monte Carlo step separation Δ is defined as,

$$\rho_O(\Delta) = \frac{\Gamma_O(\Delta)}{\Gamma_O(0)},$$
$$\Gamma_O(\Delta) = \big\langle (O_n - \langle O \rangle)(O_{n+\Delta} - \langle O \rangle) \big\rangle, \tag{1}$$

where the average runs over the full chain and $\Gamma_O(0)$ is the variance of O. The associated integrated autocorrelation time

$$t_{\mathrm{int},O} = \frac{1}{2} + \sum_{\Delta=1}^{\infty} \rho_O(\Delta) \tag{2}$$

controls the effective statistics of the chain: the variance of the sample mean is

$$\mathrm{Var}\big[\bar{O}\big] = \frac{2t_{\mathrm{int},O}}{N}\,\Gamma_O(0), \qquad N_{\mathrm{eff}} = \frac{N}{2t_{\mathrm{int},O}}, \tag{3}$$

so $2t_{\mathrm{int},O}$ consecutive configurations are required per effectively independent one. On a finite sized lattice integrated autocorrelation time is approximately $t_{\mathrm{int}} \sim L^z$, where the dynamical critical exponent for HMC applied to scalar field theories is $z \approx 2$[1] so autocorrelation time grows exponentially with lattice size. This *critical slowing down* is the central obstacle HMC methods face, and it motivates samplers that draw statistically independent configurations. For an independent sampler we have $\rho_O(\Delta) = 0$ for all $\Delta \geq 1$, so $t_{\mathrm{int},O} = \frac{1}{2}$ and $N_{\mathrm{eff}} = N$.

* Correspondence: dennis.grinwald@tu-berlin.de
† Correspondence: a.singha@tu-berlin.de
‡ Correspondence: nakajima@tu-berlin.de

II. SCORE-BASED DIFFUSION MODELS

Score based diffusion models are a class of generative models that generate samples by iteratively noising training data samples and learning how to de-noise them during inference, starting from pure noise.

We use the classical diffusion time convention throughout the paper: the diffusion time runs over $t \in [0,1]$ with

$$\begin{aligned} t = 0: &\;\; \boldsymbol{\phi}(0) \sim p \propto e^{-S} \quad \text{(target distribution)},\\ t = 1: &\;\; \boldsymbol{\phi}(1) \sim p_1 = \mathcal{N}(\mathbf{0}, \mathbf{I}) \quad \text{(Gaussian prior)}. \end{aligned} \tag{4}$$

Noising therefore runs *forward* in t, from a field configuration at $t = 0$ to the prior at $t = 1$, and generation runs backward, from $t = 1$ down to $t = 0$. The noising process is the linear Itô SDE:

$$d\boldsymbol{\phi}(t) = f(t)\,\boldsymbol{\phi}(t)\,dt + g(t)\,d\mathbf{w}(t), \qquad \boldsymbol{\phi}(0) \sim p, \tag{5}$$

where $f(t)$ is an affine drift, $g(t)$ is the diffusion coefficient, and $\mathbf{w}$ is a Wiener process. Conditioned on its starting point $\boldsymbol{\phi}(0)$, this process has a closed-form Gaussian marginal, i.e. the *perturbation kernel*

$$\begin{aligned} p\big(\boldsymbol{\phi}(t) \,\big|\, \boldsymbol{\phi}(0)\big) &= \mathcal{N}\big(\boldsymbol{\phi}(t);\, a(t)\,\boldsymbol{\phi}(0),\, b^2(t)\,\mathbf{I}\big),\\ a(0) = 1,\; b(0) = 0,\; &a(1) \to 0,\; b(1) \to 1, \end{aligned} \tag{6}$$

which we can sample directly, i.e. without simulation, during training. The drift and diffusion coefficients are recovered from (a, b) as

$$\begin{aligned} f(t) &= \frac{d}{dt} \log a(t),\\ g(t) &= \sqrt{2\frac{b(t)}{a(t)}\big(a(t)\dot{b}(t) - \dot{a}(t)b(t)\big)}. \end{aligned} \tag{7}$$

Classical examples of such SDEs and their derived perturbation kernels are the VP/VE-SDE [2]. As [3] show, one can also specify the kernel coefficients through their signal-to-noise ratio and total variance,

$$\begin{aligned} \mathrm{SNR} &:= \gamma^2(t) = \frac{a^2(t)}{b^2(t)},\\ \mathrm{TV} &:= \tau^2(t) = a^2(t) + b^2(t), \end{aligned} \tag{8}$$

with γ^2 *decreasing* in t, since $t = 0$ is the clean field. This allows for novel noise schedules including ones that follow a VE-SDE signal-to-noise-ratio and VP-like total-variance, which we will use in the paper and which are further described in the Letter.

III. DATA-FREE DIFFUSION SAMPLERS

Iterated Denoising Energy Matching (iDEM) [4] estimates the marginal score by Monte Carlo. Given $\boldsymbol{\phi}(t)$ it draws K proposals $\boldsymbol{\phi}^{(k)}(0|t)$ for the field endpoint from the perturbation kernel and forms the self-normalized importance-weighted average of energy gradients,

$$\hat{\mathbf{s}}_K\big(\boldsymbol{\phi}(t)\big) = -\sum_{k=1}^{K} \pi_k \, \nabla S\big[\boldsymbol{\phi}^{(k)}(0|t)\big],$$
$$\pi_k = \frac{\exp\big(-S[\boldsymbol{\phi}^{(k)}(0|t)]\big)}{\sum_{j=1}^{K} \exp\big(-S[\boldsymbol{\phi}^{(j)}(0|t)]\big)}, \tag{9}$$

which is consistent as $K \to \infty$. Training alternates two loops around a replay buffer $\mathcal{B}$: the outer loop integrates the reverse SDE under the current score model and writes the resulting endpoints into $\mathcal{B}$, and the inner loop draws $\boldsymbol{\phi}(0) \sim \mathcal{B}$ and t, forms $\boldsymbol{\phi}(t)$, and regresses $\mathbf{s}_{\boldsymbol{\theta}}$ onto Eq. (9). The weights are the difficulty: π_k is a softmax over K action values, so in high dimension it concentrates on a single proposal and the estimator degenerates, with a cost of K action evaluations *per training point*. This variance is the reason we do not build on iDEM directly.

Adjoint Sampling [5] removes the importance weights entirely by casting sampling as stochastic optimal control rather than score estimation. It builds on Adjoint Matching [6], which turns the control problem into a *moving* regression: instead of differentiating through an SDE solution, the control is regressed onto the adjoint state, and when the base process has zero drift that adjoint collapses to the terminal gradient $\nabla r(\boldsymbol{\phi}(0)) = \nabla \log \frac{p_0^{\text{base}}(\boldsymbol{\phi}(0))}{p(\boldsymbol{\phi}(0))}$, constant along the trajectory. The loss then depends on the pair $(\boldsymbol{\phi}(t), \boldsymbol{\phi}(0))$ only, and the *Reciprocal* Adjoint Matching objective of Ref. [5] exploits this: $\boldsymbol{\phi}(0)$ is drawn by simulating the current control, while $\boldsymbol{\phi}(t)$ is drawn from the base process pinned at that endpoint, $p_{t|0}^{\text{base}}$, which is Gaussian in closed form. The regression target is a *single* energy gradient at the endpoint, so no proposals and no weights appear anywhere.

Concretely, the algorithm alternates two loops around a buffer, similar to iDEM. The outer loop simulates the controlled SDE to obtain endpoints $\boldsymbol{\phi}^{(i)}(0)$ and evaluates ∇S *once* per endpoint, storing the pairs $(\boldsymbol{\phi}^{(i)}(0), \nabla S^{(i)})$ in $\mathcal{B}$. The inner loop then performs many gradient steps on [5] by resampling t and $\boldsymbol{\phi}(t) \sim p_{t|0}^{\text{base}}$ from those stored pairs. Like iDEM, it never simulates trajectories nor re-evaluates the action, which can be an expensive operation. Freezing the endpoint distribution while converging the inner regression, then refreshing it, is a fixed-point iteration whose fixed point is the optimal control [5]. The noise schedule enters through the base process (f, g, p_1), which must be chosen so that the memoryless condition holds. Our method can be seen as a certain instantiation of this as described in the section above.

Diffusion model based data-free methods. Data-free diffusion models are an active area of research, but few are applicable to LFT. iDEM [4] and Adjoint Sampling [5], are prototypical examples. The former pays K action evaluations per training point through the importance-weighted target (9), the latter one per buffer sample. Overall, data-free diffusion based methods for LFT are very little explored, and related methods are hardly applicable to LFT. This is why we develop our own method.

IV. LOSS, WEIGHTING AND TIME SAMPLING

The network is fitted by weighted least squares,

$$\mathcal{L}(\boldsymbol{\theta}) = \mathbb{E}_{\boldsymbol{\phi}(0)\sim\mathcal{B},\, t,\, \boldsymbol{\epsilon}}\Big[w(t)\,\big\|\mathbf{s}_{\boldsymbol{\theta}}(\boldsymbol{\phi}(t), t) - \hat{\mathbf{s}}_{\text{CVSI}}\big\|^2\Big]. \tag{10}$$

The target depends on $(\boldsymbol{\phi}(0), \boldsymbol{\epsilon})$ whereas the model sees only $\boldsymbol{\phi}(t)$, so the minimiser of Eq. (12) is the conditional expectation $\mathbb{E}[\hat{\mathbf{s}}_{\text{CVSI}} \,|\, \boldsymbol{\phi}(t)]$. The weight $w(t) > 0$ acts pointwise in t and leaves that minimiser unchanged. Because the blend (13) has already removed the $1/a$ and $1/b$ prefactors, the target has no strongly t-dependent scale left for the weight to correct, and we use $w(t) \equiv 1$; we report this rather than tuning it, and did not find a weighting that improved on it.

Times are drawn uniformly, $t \sim \mathcal{U}[t_{\min}, t_{\max}]$ with $t_{\min} = 0.01$ and $t_{\max} = 0.99$. The truncation is a numerical guard at both ends: $b \to 0$ as $t \to 0$ and $a \to 0$ as $t \to 1$, and although the blended target (13) stays finite there, the individual identities entering the clip (14) do not.

V. THE ITERATIVE TRAINING LOOP

The two ingredients above are coupled: the endpoints in Eq. (12) are drawn from the model that Eq. (12) is training. Writing $\mathcal{T}$ for the map that takes a score model to the model obtained by (i) drawing a set of configurations with it and (ii) regressing on targets built from those configurations, training iterates

$$\boldsymbol{\theta}_{i+1} = \mathcal{T}(\boldsymbol{\theta}_i), \tag{11}$$

and each round takes E gradient steps rather than solving the regression exactly, so Eq. (13) is an inexact iteration. Concretely, one round is:

(i) Buffer refresh (outer loop). We draw N_{gen} configurations $\boldsymbol{\phi}(0)$ by integrating the SDE backward in t, from the prior at $t = 1$ down to $t = 0$, with the current score model, and write them into a replay buffer $\mathcal{B}$, replacing its previous contents. This is the only place where trajectory integration occurs. Besides generation we compute the actions $-S(\boldsymbol{\phi}^{(i)}(0))$ for each generated point and store them in the buffer for reuse. We also re-estimate σ^2_{data} through Eq. (11), once the warm-up is over, so the blend coefficient $c^*(t)$ tracks the buffer from one round to the next.

(ii) Target construction and regression (inner loop). For each step we draw $\boldsymbol{\phi}(0) \sim \mathcal{B}$, a time t and fresh noise $\boldsymbol{\epsilon}$, form $\boldsymbol{\phi}(t)$ in one shot through Eq. (5), build the target (14) from $\nabla S[\boldsymbol{\phi}(0)]$ and $\boldsymbol{\epsilon}$, and take a gradient step on Eq. (12). No trajectory is simulated and no gradient is propagated through an SDE solve, so the inner loop is simulation-free.

After E steps the buffer is refreshed and the round repeats; the procedure is summarised in Algorithm 1. We maintain an exponential moving average of the parameters and use it both for buffer generation and for evaluation. Since the buffer is regenerated from the averaged model, this damps the iteration (13) rather than merely smoothing the output.

VI. OBSERVABLE DEFINITIONS

(Absolute) Magnetization For a single field configuration $\boldsymbol{\phi} = \{\phi_{\mathbf{x}}\}_{\mathbf{x}\in\Lambda_L}$ the magnetization and the absolute magnetization are the lattice averages

$$m[\boldsymbol{\phi}] = \frac{1}{|\Lambda_L|}\sum_{\mathbf{x}\in\Lambda_L}\phi_{\mathbf{x}}, \qquad |m|[\boldsymbol{\phi}] = \Big|\frac{1}{|\Lambda_L|}\sum_{\mathbf{x}\in\Lambda_L}\phi_{\mathbf{x}}\Big|, \tag{12}$$

with $|\Lambda_L| = L^D$ the number of lattice sites. We compare the full histograms of m and $|m|$ under the learned sampler against the HMC reference, as well as the ensemble averages $\langle m\rangle$ and $\langle|m|\rangle$. Since the action is invariant under $\boldsymbol{\phi} \to -\boldsymbol{\phi}$, $\langle m\rangle = 0$ in the symmetric ensemble and $\langle|m|\rangle$ is the order parameter that distinguishes the symmetric ($\langle|m|\rangle \to 0$) from the broken phase ($\langle|m|\rangle > 0$); see Fig. 8. Reporting both is informative: $\langle m\rangle \approx 0$ alone only certifies that the two modes are visited with equal weight, whereas agreement of the full $p(m)$ additionally certifies that each mode has the correct shape.

VII. FULL FREE-ENERGY RESULTS AND OBSERVABLES

The observable table uses 100k samples using the same evaluation protocol as discussed in the main paper. Concretely, we apply 100 ODE steps for sample generation and log-likelihood computation and integrate over the time interval $t \in []0.01, 0.99]$ using the Tsit5 solver.

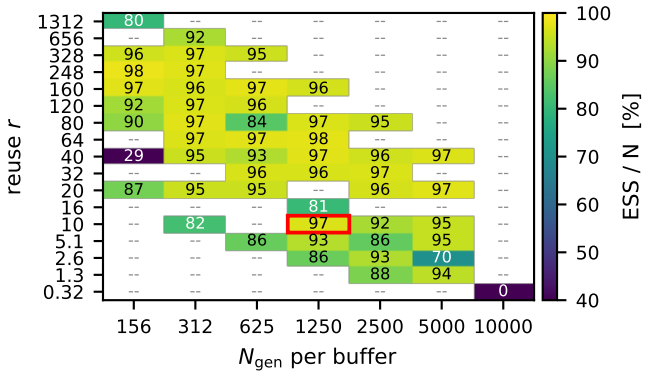


FIG. 1. Sample-reuse ablation ($L = 8$, critical phase): ESS vs. buffer size N_{gen} and reuse factor $r = \text{SPR} \cdot B/N_{\text{gen}}$. Baseline ($r \approx 10.2$) boxed.

VIII. SAMPLE-REUSE ABLATION AND TRAINING OPTIMIZATION

Figure 3 investigates how sample reuse can be exploited to further optimize the training loop and beat the computational cost of the benchmark. In our iterative loop, N_{gen} configurations are drawn by reverse-SDE integration during each outer round and stored in the replay buffer $\mathcal{B}$. The model then performs SPR = epochs_per_round $\times$ $n_{\text{steps_per_epoch}}$ gradient steps in the inner loop at batch size $B = 128$, resulting in an average reuse of

$$r = \frac{\text{SPR} \cdot B}{N_{\text{gen}}} \tag{13}$$

passes per generated configuration before buffer eviction. Rather than requiring frequent, costly buffer regenerations (small r), the sampler maintains high ESS ($> 95\%$) across a wide plateau up to $r \approx 123$ ($N_{\text{gen}} = 312, \text{SPR} = 300$). This significantly amortizes the trajectory-integration overhead of data-free training. Only when reuse is pushed excessively high or the buffer size is severely constricted does performance degrade due to stale or under-dispersed buffer configurations.

IX. LIKELIHOOD COMPUTATION

Exact likelihood computation. The generative SDE of the control parameterisation of Ref. [5] has the

TABLE I. Observables from HMC, raw sampler, and reweighted samples ($w_i \propto e^{-W_i}$). Errors in parentheses: HMC uses the Γ-method with automatic windowing ($S = 2$); sampler estimates use independent-sample joint-moment covariance propagation. Model sample counts match the main free-energy table; each HMC reference uses 200,000 configurations. The $L = 16$ symmetric evaluation uses $\sigma_{\min} = 0.001$.

		$\langle\lvert m\rvert\rangle$			$\chi_{\lvert M\rvert}$			χ_M			U_4			
L	phase	HMC	raw	rew	HMC	raw	rew	HMC	raw	rew	HMC	raw	rew	wESS
4	symmetric	0.580(1)	0.577(2)	0.580(2)	2.433(7)	2.53(1)	2.46(1)	7.81(2)	7.86(4)	7.84(4)	0.217(2)	0.195(4)	0.213(3)	0.998
	critical	1.041(2)	1.016(2)	1.038(2)	4.24(1)	4.31(1)	4.24(1)	21.56(5)	20.82(5)	21.49(6)	0.4548(9)	0.440(1)	0.4538(10)	0.988
	broken	1.399(2)	1.366(2)	1.396(2)	4.00(2)	4.24(3)	4.05(2)	35.31(6)	34.08(9)	35.24(9)	0.5546(5)	0.5417(8)	0.5528(7)	0.989
8	symmetric	0.3160(7)	0.307(1)	0.313(1)	3.31(1)	3.27(2)	3.29(2)	9.70(4)	9.32(6)	9.58(6)	0.114(3)	0.083(5)	0.114(5)	0.99
	critical	0.917(2)	0.864(1)	0.912(1)	9.68(4)	10.62(4)	9.73(4)	63.5(2)	58.4(1)	62.9(1)	0.514(1)	0.4787(9)	0.5110(7)	0.967
	broken	1.4452(10)	1.433(1)	1.447(1)	4.60(4)	5.59(5)	4.58(4)	138.0(3)	137.0(2)	138.5(2)	0.6300(2)	0.6239(3)	0.6303(2)	0.974
16	symmetric	0.1587(4)	0.1461(5)	0.1575(6)	3.59(2)	3.08(2)	3.54(3)	10.04(4)	8.55(5)	9.89(7)	0.033(4)	0.018(7)	0.035(8)	0.961
	critical	0.814(2)	0.621(1)	0.814(1)	23.1(2)	28.60(9)	23.15(10)	192.3(9)	127.4(3)	192.6(5)	0.553(1)	0.423(1)	0.5523(6)	0.556
	broken	1.4557(4)	1.3857(9)	1.4549(9)	4.15(2)	9.8(1)	4.08(4)	4.15(2)	501.3(5)	545.9(7)	0.65693(4)	0.6467(2)	0.65710(9)	0.558

probability-flow ODE

$$\frac{d\boldsymbol{\phi}(t)}{dt} = \underbrace{f(t)\,\boldsymbol{\phi}(t) - g(t)\,\mathbf{u}_t(\boldsymbol{\phi}(t)) + \tfrac{1}{2}g^2(t)\,\mathbf{s}_{\boldsymbol{\theta}}(\boldsymbol{\phi}(t),t)}_{=:\ \mathbf{f}_{\boldsymbol{\theta}}(\boldsymbol{\phi}(t),t)}, \tag{14}$$

with the same f, g as in Eqs. (4)–(7); under the parameterisation [5] this is $\mathbf{f}_{\boldsymbol{\theta}} = f\boldsymbol{\phi} - \frac{1}{2}g^2\mathbf{s}_{\boldsymbol{\theta}}$, and generation integrates it from the prior at $t = 1$ down to $t = 0$. We can use the instantaneous change of variables formula [7] to compute the log-likelihood of a generated configuration $\boldsymbol{\phi}(0)$ under our trained sampler's distribution $q(\cdot)$ by integrating

$$\log q(\boldsymbol{\phi}(0)) = \log p_1(\boldsymbol{\phi}(1)) + \int_0^1 \nabla\cdot\mathbf{f}_{\boldsymbol{\theta}}(\boldsymbol{\phi}(t),t)\,dt, \tag{15}$$

where $\nabla\cdot\mathbf{f}_{\boldsymbol{\theta}} = \mathrm{Tr}(\mathbf{J}_{\mathbf{f}_{\boldsymbol{\theta}}})$ is the trace of the Jacobian of $\mathbf{f}_{\boldsymbol{\theta}}$ and $p_1 = \mathcal{N}(\mathbf{0}, \mathbf{I})$ is the prior at $t = 1$.

Approximate likelihood computation. In settings where computing the trace of the Jacobian is too costly, we can instead use an approximation as proposed by [8, 9]. In particular, the Skilling-Hutchinson trace estimator computes:

$$\nabla\cdot\mathbf{f}_{\boldsymbol{\theta}}(\boldsymbol{\phi}(t),t) = \mathbb{E}_{p(\mathbf{v})}\big[\mathbf{v}^\top\nabla\mathbf{f}_{\boldsymbol{\theta}}\,\mathbf{v}\big], \tag{16}$$

where $\mathbf{v} \sim \mathcal{N}(\mathbf{0}, \mathbf{I})$ is a probe vector, kept distinct from the forward-kernel noise $\boldsymbol{\epsilon}$. The VJP $\mathbf{v}^\top\nabla\mathbf{f}_{\boldsymbol{\theta}}$ can be efficiently computed using reverse mode auto-diff at approx. the same cost as evaluating $\mathbf{f}_{\boldsymbol{\theta}}$. Thus, we can sample $\mathbf{v} \sim p(\mathbf{v})$ and compute an efficient unbiased estimate of $\nabla\cdot\mathbf{f}_{\boldsymbol{\theta}}$ using $\mathbf{v}^\top\nabla\mathbf{f}_{\boldsymbol{\theta}}(\boldsymbol{\phi}(t),t)\,\mathbf{v}$.

X. HYPERPARAMETERS

Table **??** gives the nominal main-method setup and evaluation protocol.

XI. ESTIMATOR-SPECIFIC HYPERPARAMETERS AND INTEGRATION

This section documents the main-result evaluation campaign, including its matched-architecture replacement checkpoints. Completed pools are promoted separately; a configuration listed here does not imply that its evaluation has finished. Separate architecture experiments are not included in the main-result tables.

Noise schedules and loss weighting. For every row in Table **??**,

$$\sigma(t) = \sigma_{\min}\left(\frac{\sigma_{\max}}{\sigma_{\min}}\right)^{t}. \tag{17}$$

With $\boldsymbol{\phi}_t = a(t)\boldsymbol{\phi}_0 + b(t)\boldsymbol{\epsilon}$, the variance-exploding (VE) schedule uses $a(t) = 1$, $b(t) = \sigma(t)$, while the constant-total-variance (CV) schedule uses

$$a(t) = \frac{1}{\sqrt{1+\sigma(t)^2}}, \qquad b(t) = \frac{\sigma(t)}{\sqrt{1+\sigma(t)^2}}. \tag{18}$$

The endpoint parameters specify the schedule on $[0, 1]$; the numerical flows are truncated to $[0.01, 0.99]$. Thus $\sigma_{\min}$ is not the interpolation time cutoff. The scalar training loss multiplier is defined by

$$\mathcal{L} = \mathbb{E}\Big[w(t)\,\|\mathbf{s}_\theta(\boldsymbol{\phi}_t, t) - \hat{\mathbf{s}}(\boldsymbol{\phi}_t, t)\|^2\Big]. \tag{19}$$

Here $w(t)$ is either 1 or $\sigma(t)^2$ as tabulated. It is distinct from importance weights used inside an IW target and from the normalized importance weights used for observable estimation. In particular, CVSI (ours) uses the constant loss multiplier.

UNet implementation. The score network uses the UNet architecture [**?** **?**] with the size-specific settings in Table **??**. The diffusion time t is encoded with random Fourier features [**?**], $t \mapsto [t, \sin(2\pi\boldsymbol{\omega}t), \cos(2\pi\boldsymbol{\omega}t)]$ with a fixed random-Gaussian frequency vector $\boldsymbol{\omega}$, and mapped by a two-layer MLP (GELU) to a time embedding $\mathbf{e}_t$ of dimension $4C$. Each residual block consists of two 3×3 convolutions with RMSNorm and SiLU activations; the time embedding is injected as a per-channel affine (FiLM) modulation $\mathbf{h} \mapsto \mathbf{h}\,(1 + \boldsymbol{\gamma}_{\mathrm{F}}) + \boldsymbol{\delta}$ obtained from a linear projection of the time vector.

Architecture and symmetry. Table **??** gives the architecture shared by all methods at each size. Width denotes the base number of feature channels, not the number of lattice sites. There are seven residual blocks, each containing two 3×3 convolutions, as well as two 3×3 stage transitions. Skip concatenation doubles the input channels before three residual blocks; these blocks reduce the channels back to the base width. Three attention modules have two heads with 32 channels per head; each uses a 1×1 projection to 192 query/key/value channels and a 1×1 projection from 64 channels back to the base width. The final 1×1 convolution returns a single score field. Spatial feature maps retain the lattice resolution. Circular padding implements periodic boundaries for the spatial convolutions. The model enforces odd score symmetry through

$$\mathbf{s}_\theta^{\mathbb{Z}_2}(\boldsymbol{\phi}, t) = \tfrac{1}{2}\big[\mathbf{s}_\theta(\boldsymbol{\phi}, t) - \mathbf{s}_\theta(-\boldsymbol{\phi}, t)\big]. \tag{20}$$

This is the model-level wrapper used in both training and evaluation.

[1] S. Duane, A. D. Kennedy, B. J. Pendleton, and D. Roweth, Physics letters B **195**, 216 (1987).

[2] Y. Song, J. Sohl-Dickstein, D. P. Kingma, A. Kumar, S. Ermon, and B. Poole, in *International Conference on Learning Representations* (2021).

[3] K. Kahouli, W. Ripken, S. Gugler, O. T. Unke, K.-R. Müller, and S. Nakajima, arXiv preprint arXiv:2502.08598 (2025).

[4] T. Akhound-Sadegh, J. Rector-Brooks, A. J. Bose, S. Mittal, P. Lemos, C.-H. Liu, M. Sendera, S. Ravanbakhsh, G. Gidel, Y. Bengio, N. Malkin, and A. Tong, in *International Conference on Machine Learning (ICML)* (2024).

[5] A. J. Havens, B. K. Miller, B. Yan, C. Domingo-Enrich, A. Sriram, D. S. Levine, B. M. Wood, B. Hu, B. Amos, B. Karrer, X. Fu, G.-H. Liu, and R. T. Q. Chen, in *Proceedings of the 42nd International Conference on Machine Learning*, Proceedings of Machine Learning Research, Vol. 267, edited by A. Singh, M. Fazel, D. Hsu, S. Lacoste-Julien, F. Berkenkamp, T. Maharaj, K. Wagstaff, and J. Zhu (PMLR, 2025) pp. 22204–22237.

[6] C. Domingo i Enrich, M. Drozdzal, B. Karrer, and R. T. Chen, in *International Conference on Learning Representations*, Vol. 2025 (2025) pp. 53791–53846.

[7] R. T. Q. Chen, Y. Rubanova, J. Bettencourt, and D. K. Duvenaud, Advances in Neural Information Processing Systems **31** (2018).

[8] J. Skilling, in *Maximum Entropy and Bayesian Methods: Cambridge, England, 1988* (Springer, 1989) pp. 455–466.

[9] M. F. Hutchinson, Communications in Statistics-Simulation and Computation **18**, 1059 (1989).

	Setting	Value
Target	score identity	CVSI (clipped TSI/DSI blend)
	clip $\|\mathbf{s}\|$	70 (each of TSI, DSI, before blending)
	MC proposals per sample	500
	σ^2_{data}	1.0 during warm-up, then Eq. (11)
	warm-up R_{warm}	100 rounds
	clamp $[\sigma^2_-, \sigma^2_+]$	[0.25, 10.0]
Schedule	type	$a = 1/\sqrt{1+\sigma^2}$, $b = \sigma/\sqrt{1+\sigma^2}$
	total variance	$\tau^2 \equiv 1$
	loss weight	$w(t) \equiv 1$
	time range	$t \in [0.01, 0.99]$
Network	architecture	UNet, circular padding, $\mathbb{Z}_2$-odd
	base width C	8 ($L \in \{4, 8\}$), 16 ($L = 16$)
	stages	1 (no down/upsampling)
	stem kernel	3 ($L = 4$), 7 ($L \in \{8, 16\}$)
	parameters	25,041 / 80,409
Optimisation	optimiser	AdamW, lr 10^{-3}, plateau decay ($\times 0.1$, patience 10)
	batch size	128
	gradient clipping	global norm 1.0
	EMA decay	0.999
Fixed point	buffer size	1250 (fully replaced each round)
	steps per refresh	100
	rounds	1000 ($L \in \{4, 8\}$), 3000 ($L = 16$)
	buffer sampler	run-dependent; see estimator-specific supplement table
Evaluation	sampler	probability-flow ODE, Tsit5, 100 steps
	$\log q$	exact trace, Tsit5, 100 steps
	precision	float32 generation; float64 likelihood
	integration interval	$[0.01, 0.99]$ (both directions)
	samples	target 5×10^4; current counts in Table I
Reference	method	thermodynamic integration in κ
	grid	200 points, cubically clustered toward κ
	statistics	10^5 configurations per point

TABLE II. Main-method settings.

L	Phase	Estimator	Schedule	$\sigma_{\min}$	$\sigma_{\max}$	Training $w(t)$
4	symmetric	CVSI (ours)	CV	0.01	50	1
4	critical	CVSI (ours)	CV	0.01	25	1
4	critical	d-w	VE	0.01	10	$\sigma(t)^2$
4	critical	d-TSI	VE	0.01	10	1
4	critical	d-CVSI	CV	0.01	50	1
4	critical	TSI	VE	0.01	10	$\sigma(t)^2$
4	critical	IW-CVSI	CV	0.01	5	1
4	critical	IW-TSI	CV	0.01	5	1
4	broken	CVSI (ours)	CV	0.01	100	1
8	symmetric	CVSI (ours)	CV	0.001	50	1
8	critical	CVSI (ours)	CV	0.01	50	1
8	critical	d-DSI	VE	0.01	50	$\sigma(t)^2$
8	critical	d-TSI	VE	0.01	10	1
8	critical	d-CVSI	CV	0.01	50	1
8	critical	TSI	VE	0.01	10	$\sigma(t)^2$
8	critical	IW-CVSI	CV	0.01	5	1
8	critical	IW-TSI	CV	0.01	5	1
8	broken	CVSI (ours)	CV	0.01	250	1
16	symmetric	CVSI (ours)	CV	0.001	10	1
16	critical	CVSI (ours)	CV	0.01	100	1
16	critical	d-DSI	VE	0.01	50	$\sigma(t)^2$
16	critical	d-TSI	VE	0.01	10	1
16	critical	d-CVSI	CV	0.01	50	$b(t)^2$
16	critical	TSI	VE	0.01	10	$\sigma(t)^2$
16	critical	IW-CVSI	CV	0.01	100	1
16	critical	IW-TSI	CV	0.01	100	1
16	broken	CVSI (ours)	CV	0.001	100	$\sigma(t)^2$

TABLE III. Schedule settings of all 27 configurations in the main-result evaluation campaign, ordered by lattice size and phase. CV and VE are defined in the text.